

Documentación Técnica - Módulo de Administración y Roles (Actualización #4)

Esta sección detalla la implementación del Control de Acceso Basado en Roles (RBAC) y la interfaz de administración para la gestión de usuarios, aplicando principios de "Defensa en Profundidad" y asegurando las variables de entorno.

1. Control de Acceso Reactivo en la Interfaz (Dashboard)

Para garantizar que las funciones administrativas estén estrictamente ocultas a usuarios estándar, se implementó un renderizado condicional en la capa de la vista (`DashboardScreen`).

- **Inyección de Dependencias:** Se modificó el enrutador central (`AppRoutes`) para inyectar el `AuthController` directamente en el constructor del `DashboardScreen` .
- **Validación de Estado:** El botón de acceso a la gestión de usuarios (CRUD) evalúa en tiempo real el getter `authController.usuarioActual?.esAdmin` . Si el token/sesión actual no posee el rol de `admin` , el punto de acceso en el `AppBar` simplemente no se renderiza, minimizando la superficie de ataque en la UI.

2. Inyección Segura de Privilegios (Registro de Administradores)

Para resolver el dilema de escalado de privilegios sin comprometer el código fuente (evitando dejar credenciales o códigos "quemados" / hardcoded), se integró el paquete `flutter_dotenv` .

- El proceso de registro (`RegisterScreen`) expone un campo opcional para un "Código Admin".
- En el controlador de la vista, se intercepta este input y se compara en memoria contra la variable de entorno `SECRET_ADMIN_CODE` leída directamente del archivo `.env` local.
- Si el desafío criptográfico coincide, el objeto `UsuarioModel` es instanciado con el atributo `rol: 'admin'` ; en caso contrario, se asigna el rol de mínima autoridad (`rol: 'operador'`) por defecto.

3. Defensa en Profundidad (Sudo Mode) en el CRUD de Usuarios

El panel de administración (`AdminUsersScreen`) permite al administrador visualizar, editar (escalar privilegios) y eliminar usuarios de la base de datos local SQLite. Para prevenir borrados accidentales o ataques de sesión (ej. un administrador deja la app abierta y alguien más intenta borrar datos o darse permisos de admin), se implementó un mecanismo de reautenticación activa conocido como *Sudo Mode*:

- **Intercepción de Operaciones Críticas:** Las funciones de `actualizarDatosUsuario` y `eliminarUsuario` están bloqueadas por un `AlertDialog` con estado propio (`StatefulBuilder`).
- **Validación de Identidad:** Antes de invocar la transacción SQL de eliminación o de actualización de roles, el sistema exige que el administrador ingrese **su propia contraseña actual**.
- **Ejecución Segura:** El controlador compara el input con `widget.authController.usuarioActual?.password` . Solo si la coincidencia es exacta, se permite el acceso al método de escritura/borrado en la base de datos, notificando el resultado al usuario mediante un `SnackBar` .

4. Filtrado Dinámico en Memoria

Para optimizar el rendimiento del CRUD sin saturar el motor SQLite con consultas `LIKE` repetitivas, el buscador de la `AdminUsersScreen` implementa un filtrado en memoria sobre la propiedad `listaUsuarios` del `AuthController` . La UI reacciona mediante el `ListenableBuilder` recalculando la lista mostrada en tiempo real (`where (u) => u.nombre.contains...`) a medida que el administrador digita en el `TextField` del AppBar.